

CS6811 – Project Work

Federated Learning for Code Generation with Hallucination-Aware Fine Tuning

Team Members:

Abhinavh P (2022503059)

Prabhakara Arjun R (2022503003)

Buvanes Srivardan K (2022503037)

MENTOR

Dr. B. Thanasekhar,
Professor,
Computer Technology,
MIT

INTRODUCTION

- Large Language Models (LLMs) have remarkable capabilities in code generation ,enabling automated solutions to a wide range of software engineering tasks.
- However LLMs frequently suffer from hallucinations in code generation, producing syntactically valid but semantically incorrect programs, non-existent APIs, or logically flawed implementations.
- Existing approaches to mitigating code hallucinations primarily rely on prompt engineering, repeated sampling, or full model fine-tuning.
- Full fine-tuning is computationally expensive and impractical in low-resource or decentralized settings, and repeated sampling does not address the underlying causes and mitigation of hallucination.
- In real-world development environments, code generation systems are often deployed across multiple decentralized clients where data cannot be centrally aggregated due to privacy, ownership.
- To address these limitations we propose a hallucination-aware code generation framework that combines program analysis, targeted automatic program repair, and federated parameter-efficient fine-tuning using LoRA.

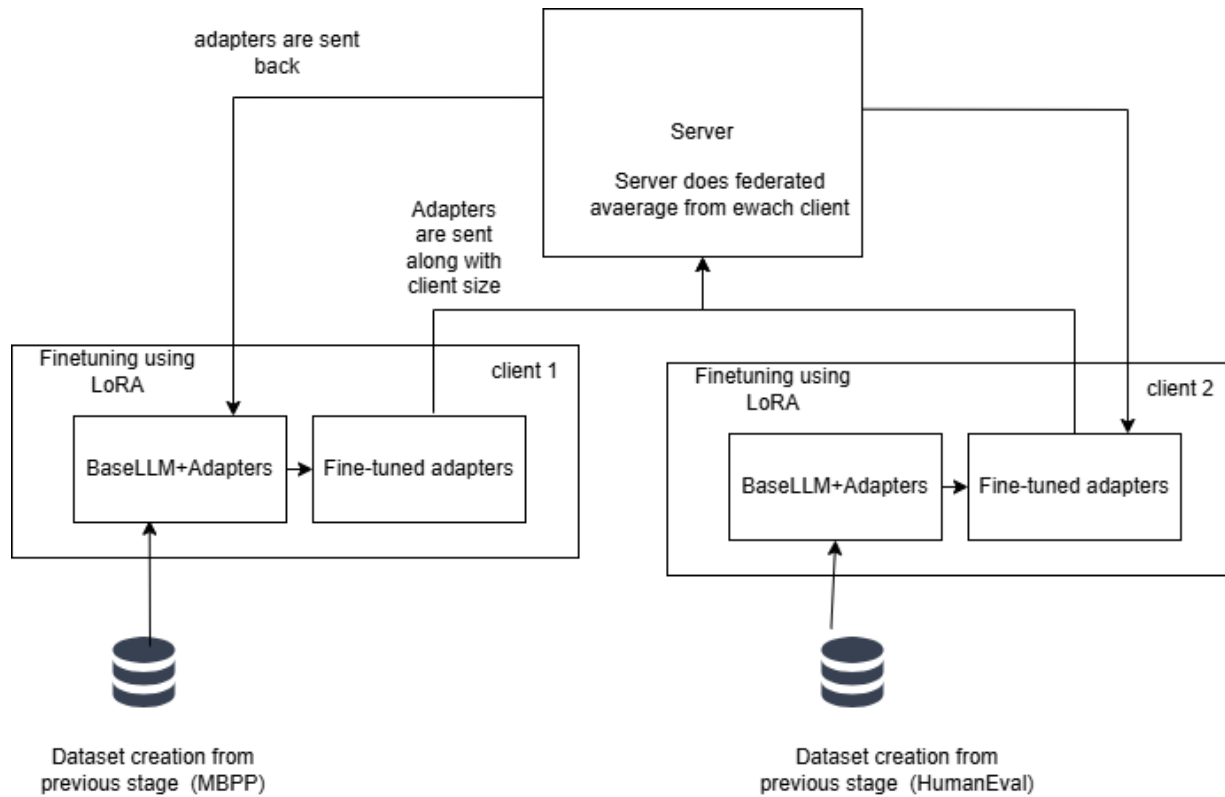
PROBLEM STATEMENT

- Code generation using large language models (LLMs) is currently hindered by hallucinations.
- Existing mitigation strategies typically rely on supervised fine-tuning or reinforcement learning from human feedback (RLHF), where models are trained to align with human preferences .
- While parameter-efficient fine-tuning methods such as LoRA reduce computational overhead compared to full fine-tuning, both approaches primarily optimize model behavior but without explicit verification of program correctness.
- As a result, these methods may improve correctness but often fail to systematically detect and correct deeper structural, semantic, and execution-level errors in generated code.
- Furthermore, in real world scenarios, high-quality code and execution feedback are distributed across multiple organizations making centralized data collection infeasible due to privacy concerns.
- Without a secure and decentralized learning mechanism, models are unable to effectively learn from diverse hallucination patterns observed across clients.
- This creates a critical gap for a unified framework that integrates explicit hallucination detection and automated program verification with privacy-preserving, parameter-efficient adaptation to improve the quality of code generation .

OBJECTIVES

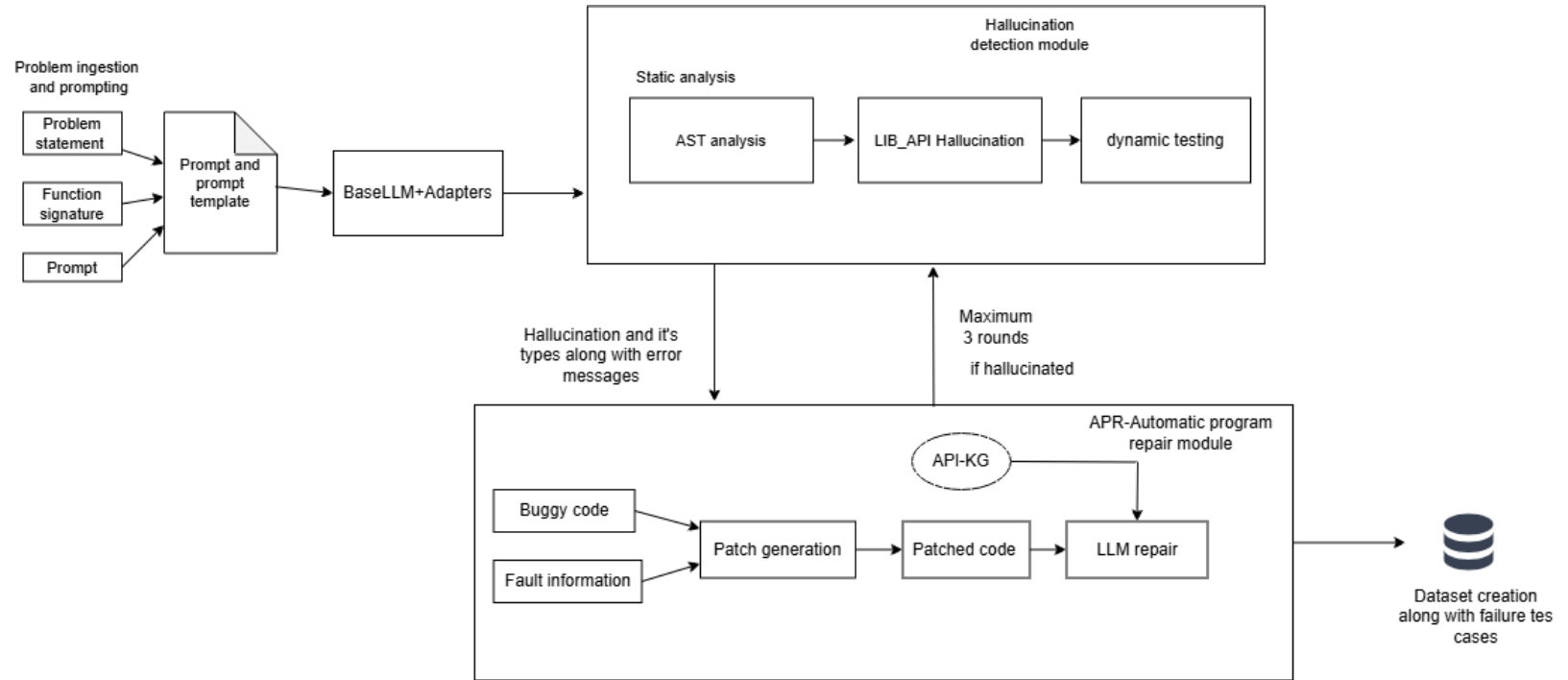
- To design a hallucination-aware code generation framework that integrates static and dynamic program analysis.
- To fine-tune large language models using parameter-efficient LoRA in a federated learning setting.
- To evaluate the proposed framework on standard code generation benchmarks and analyze hallucination reduction.
- To detect and classify different types of hallucinations in code generation.

ARCHITECTURE DIAGRAM



Federated learning + LoRA for fine-tuning

ARCHITECTURE DIAGRAM



Client side hallucination detection and mitigation framework

DATASET CREATION

- This is the final output of hallucination detection and mitigation framework.
- It contains these columns,

Dataset ,task_id , prompt ,status , ast_info ,dynamic_info ,lib_info, generated_code
patched_code , error_sources , error_types , error_lines , canonical_solution

- Prompt and canonical solution will be used for fine-tuning
- Error count and types will be used against fed weighted average.

LoRA and FINE TUNING

- Here we do supervised fine-tuning using Low-rank Adaptation technique.
- The rank chosen is 16 and alpha value to be 32 to bring down the scaling factor as 2.
- We have used the 7 projection layers Query,Key,Value,Object projection which belong to self-attention layer and gate,up,down belongs to MLP layer.
- For SFT we fine-tuned for 3 epochs with batch size 1 and gradient accumulation to be 8.
- SFT is done for all the datasets MBPP,DS1000,Humaneval and adapters are generated.
- Each fine-tuned adapters are checked for correctness and then verified for improvement in hallucination reduction.

ADAPTERS

```
=== Summary ===
```

```
Number of LoRA parameters: 504
```

```
Total elements: 29933568
```

```
Parameter names (for FedAvg): ['base_model.model.model.layers.0.self_attn.q_proj.lora_A.default.weight', 'base_model.model.model
```

```
=== Attention projections ===
```

Module	W shape	LoRA A shape	LoRA B shape	LoRA params
k_proj	[256, 2048]	[16, 2048]	[256, 16]	36,864
o_proj	[2048, 2048]	[16, 2048]	[2048, 16]	65,536
q_proj	[2048, 2048]	[16, 2048]	[2048, 16]	65,536
v_proj	[256, 2048]	[16, 2048]	[256, 16]	36,864

```
=== MLP projections ===
```

Module	W shape	LoRA A shape	LoRA B shape	LoRA params
down_proj	[2048, 11008]	[16, 11008]	[2048, 16]	208,896
gate_proj	[11008, 2048]	[16, 2048]	[11008, 16]	208,896
up_proj	[11008, 2048]	[16, 2048]	[11008, 16]	208,896

FEDERATED AVERAGE

FedAVG Function

$$w^{t+1} = \sum_{k=1}^K \frac{n_k}{N} w_k^t$$

K - client

n_k = number of training samples on client k

N – total number of samples used across all clients

Error frequency

$$F_e = \sum_{k=1}^K count_{k,e}$$

Weight definition

$$weight_e = \frac{F_e}{\sum_{e' \in errors} F_{e'}}$$

F_e = total **occurrences** of error type ' e ' across all clients

$count_{k,e}$ = number of errors of type ' e ' in client k

FEDERATED AVERAGE

Error Score

$$E_k = \sum_{e \in \text{errors}} \text{count}_{k,e} \cdot \text{weight}_e$$

For each client, the total count of each type of error along with its computed weight is accounted for

Normalisation

$$\alpha_k = \frac{E_k}{\sum_{j=1}^K E_j}$$

Augmented FedAvg

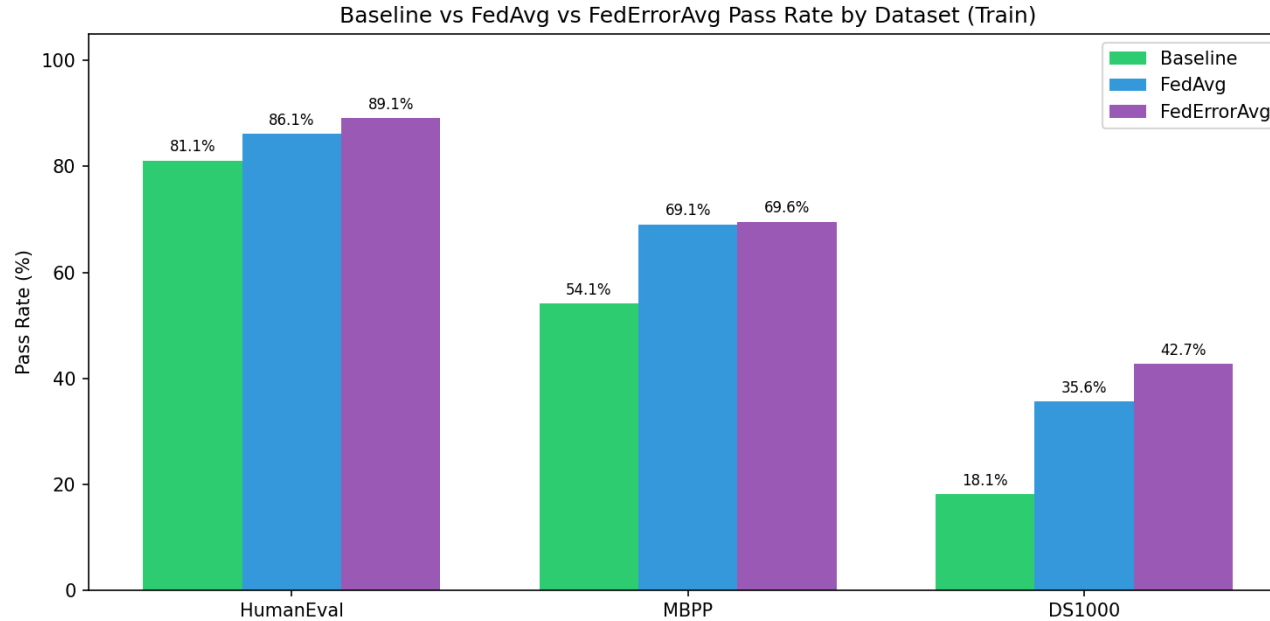
$$w^{t+1} = \sum_{k=1}^K \alpha_k w_k^t$$

w_k^t - local model weight on round t on client k

α_k - normalized weight assigned to client k

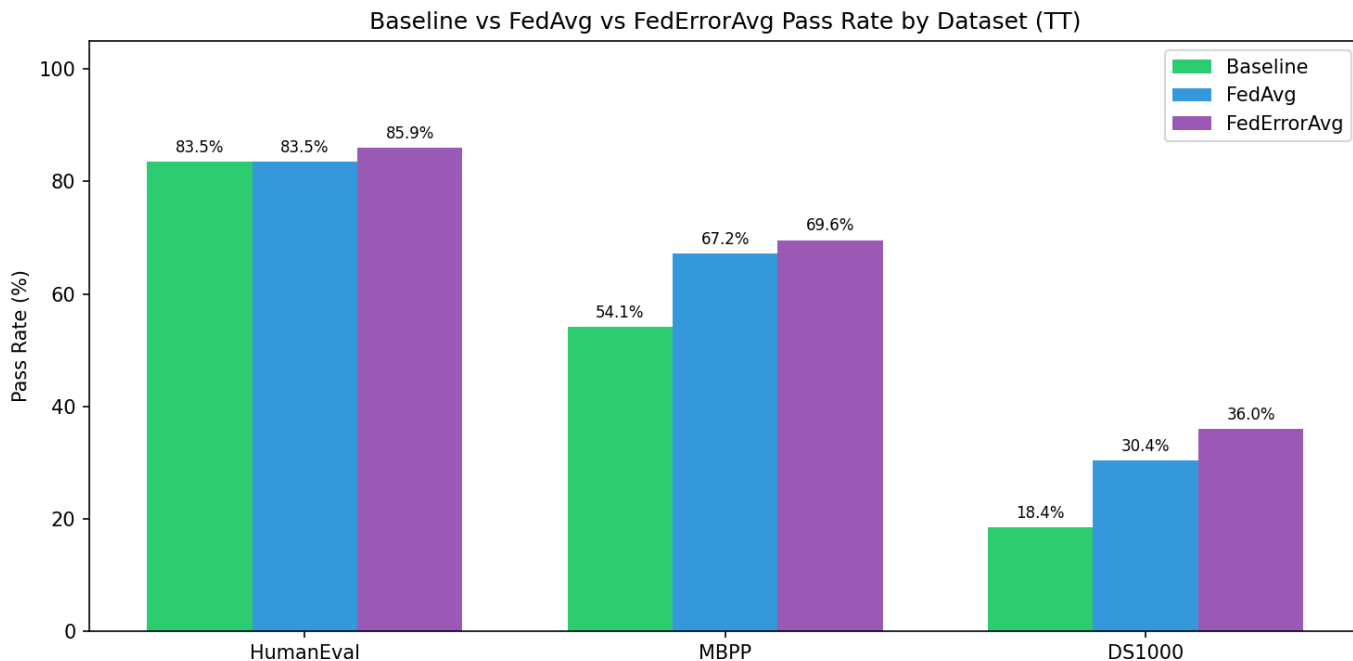
w^{t+1} - Global model weights after aggregation at round $t + 1$

RESULT AND ANALYSIS



A comparison on how federated average and error average reduces hallucinations across 3 datasets.(using train and test)

RESULT AND ANALYSIS



A comparison on how federated average and error average reduces hallucinations across 3 datasets.

RESULT AND ANALYSIS

Error type	Baseline	FedAvg	Reduction
AssertionError	5	5	0
NameError	2	2	0
SyntaxError	1	1	0

Error distribution for HumanEval
after federated average

RESULT AND ANALYSIS

Error type	Baseline	FedAvg	Reduction
AssertionError	42	52	0
IndentationError	1	0	1
IndexError	1	1	0
NameError	26	4	22
SyntaxError	17	8	9
TypeError	6	2	4
UnboundLocalError	1	0	1
ValueError	1	1	0

Error distribution for MBPP after
federated average

RESULT AND ANALYSIS

Error type	Baseline	FedAvg	Reduction
AssertionError	79	78	1
AttributeError	14	5	9
AxisError	1	2	0
FileNotFoundError	1	0	1
IndentationError	17	16	1
IndexError	1	3	0
InternalError	0	5	0
InvalidIndexError	1	0	1
KeyError	5	16	0

NameError	43	11	32
NotFittedError	2	2	0
RuntimeError	3	2	1
SyntaxError	1	0	1
TimeoutError	3	0	3
TypeError	18	15	3
ValueError	15	19	0

Error distribution for DS1000 after federated average

RESULT AND ANALYSIS

Error Type	Baseline Count	FedErrorAvg Count	Reduction
AssertionError	5	4	1
NameError	2	2	0
SyntaxError	1	1	0

Error distribution for HumanEval
after federated error average

RESULT AND ANALYSIS

Error Type	Baseline Count	FedErrorAvg Count	Reduction
AssertionError	42	51	0
IndentationError	1	0	1
IndexError	1	2	0
NameError	26	3	23
SyntaxError	17	5	12
TypeError	6	2	4
UnboundLocalError	1	0	1
ValueError	1	0	1

Error distribution for MBPP after
federated average

RESULT AND ANALYSIS

Error Type	Baseline Count	FedErrorAvg Count	Reduction
AssertionError	79	71	8
AttributeError	14	7	7
AxisError	1	1	0
FileNotFoundError	1	0	1
IndentationError	17	16	1
IndexError	1	3	0
InternalError	0	5	0
InvalidIndexError	1	0	1

KeyError	5	6	0
NameError	43	8	35
NotFittedError	2	0	2
RuntimeError	3	3	0
SyntaxError	1	4	0
TimeoutError	3	0	3
TypeError	18	16	2
ValueError	15	20	0

Error distribution for DS1000 after
federated average